

Mastering the Backtester

Ein Leitfaden zur robusten Strategieoptimierung und zur Architektur des
Backtester-Projekts

Version 1.0 - 08.07.2026

Benutzerhandbuch, Lehrbuch und technische Projektdokumentation

Inhaltsverzeichnis

Teil	Inhalt
1	Vorwort: Warum dieses Buch existiert
2	Kapitel 1: Grundlagen des Backtestings
3	Kapitel 2: Curve Fitting, Overfitting und Biases
4	Kapitel 3: Architektur des Backtester-Projekts
5	Kapitel 4: Benutzeroberfläche und Bedienlogik
6	Kapitel 5: Engine Deep Dive
7	Kapitel 6: Reporting, Scoring und Persistenz
8	Kapitel 7: Der KI-Ansatz
9	Kapitel 8: Datenversorgung mit Dukascopy und MT5 Custom Symbols
10	Kapitel 9: Parameter-Referenz
11	Kapitel 10: Robuste Optimierung in der Praxis
12	Fazit
A	Parameter-Referenz
B	Sourcecode-Modulindex
C	Kritische Klassen und Entwicklerleitfaden
D	Betriebschecklisten und Troubleshooting
E	Glossar
F	Screenshot-Galerie, Quellen und Bildnachweis

Vorwort: Warum dieses Buch existiert

Dieses Buch dokumentiert den Backtester als konkretes Softwareprojekt und fuehrt zugleich in die Denkweise robuster Strategieentwicklung ein. Es ist kein reines Benutzerhandbuch und auch kein isolierter Architekturbericht. Es verbindet die Bedienung des Programms mit den Gruenden, warum seine Pipeline so gebaut wurde: MetaTrader wird automatisiert, Optimierungen werden strukturiert, Sensitivitaet wird messbar, KI wird als Analyst und nicht als Orakel eingesetzt, und die finale Strategieauswahl wird durch eine echte Out-of-Sample-Validierung abgesichert.

Die Analyse basiert auf dem Sourcecode dieses Repositories. Zum Zeitpunkt der Bucherstellung umfasst die Hauptanwendung 79 Java-Dateien mit rund 44.007 Zeilen und die Tests 41 Java-Dateien mit rund 8.365 Zeilen. Besonders relevant sind die Pakete `engine`, `report`, `database`, `config`, `dukascopy`, `mt5` und `ui.javaafx`. Die vorhandenen Markdown-Dokumente, README-Dateien, Screenshots und Tests wurden als zusaetzliche Quellen herangezogen.

Der Text richtet sich an Nutzer, die keine Quant-Profis sind, aber trotzdem verstehen wollen, was ein Backtest leistet, was er nicht leisten kann und wie man mit einem Werkzeug wie diesem die typischen Denkfallen reduziert. Er richtet sich auch an Entwickler, die die Architektur erweitern moechten, ohne die fachlichen Schutzmechanismen aus Versehen zu unterlaufen.

Kapitel 1: Grundlagen des Backtestings

Ein Backtest ist ein kontrolliertes Gedankenexperiment mit historischen Daten. Eine Handelsregel wird so behandelt, als haette man sie in der Vergangenheit bereits gekannt, und anschliessend wird berechnet, welche Trades sie erzeugt haette. Der Nutzen liegt darin, schlechte Ideen schnell auszusortieren und gute Ideen genauer zu untersuchen, bevor echtes Kapital eingesetzt wird. Der Fehler beginnt, wenn man das Ergebnis als Vorhersage missversteht. Ein Backtest sagt nicht: Diese Strategie wird in Zukunft Geld verdienen. Er sagt nur: Unter den simulierten Annahmen haette diese Logik in diesem historischen Zeitraum so ausgesehen.

Im Backtester-Projekt wird diese Idee praktisch an MetaTrader delegiert. Die Java- Anwendung schreibt eine `tester.ini`, startet MT4 oder MT5, wartet auf den Report, kopiert die Ergebnisdateien und parst die Kennzahlen. Dadurch bleibt die eigentliche Handelssimulation bei der Plattform, die Expert Advisors ohnehin ausfuehrt. Die Backtester-Anwendung wird zur Orchestrierungsschicht: Sie standardisiert Eingaben, automatisiert Wiederholungen, sammelt Resultate und fuegt robuste Bewertung hinzu.

Gute Backtests brauchen drei Arten von Disziplin. Erstens technische Disziplin: Reports duerfen nicht veraltet sein, Prozesse duerfen nicht haengen bleiben, Parameterdateien muessen exakt zugeordnet werden. Zweitens statistische Disziplin: wenige Trades sind schwache Evidenz, ein einzelner Zeitraum ist keine Marktwahrheit, und jede zusaetzliche Optimierungsrunde erhoeht die Gefahr, Rauschen zu lernen. Drittens operative Disziplin: Ergebnisse muessen gespeichert, reproduzierbar und kommentierbar sein. Genau aus dieser Dreiteilung erklaert sich die Architektur des Projekts.

In der Praxis ist Backtesting nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhaengig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer spaeter nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufaelligen Optimierungsspur.

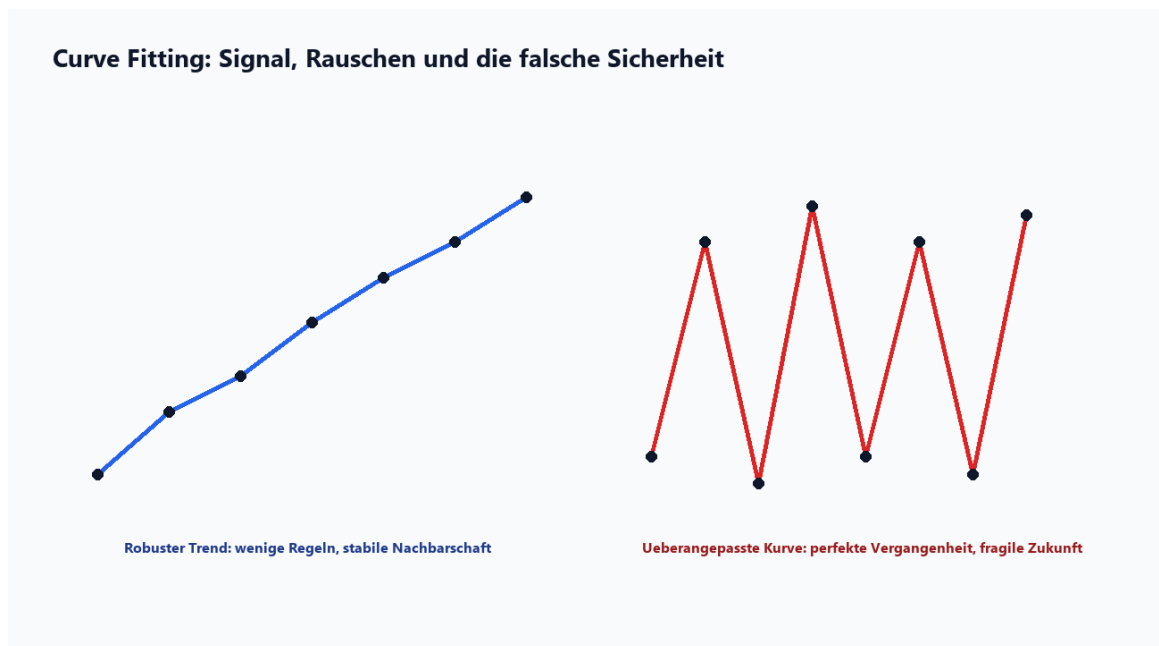
Fuer das Projekt bedeutet das konkret: `BacktestRunner`, `IniGenerator`, `ReportParser` und `DatabaseManager` bilden gemeinsam den Kern eines reproduzierbaren Einzeltests. Diese technische Entscheidung hat eine

fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberfläche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Für Einsteiger ist die wichtigste Regel: Ein Backtest ist kein Verkaufsprospekt, sondern ein Diagnoseinstrument. Eine Strategie mit Verlust im Backtest ist meistens kein Kandidat. Eine Strategie mit Gewinn im Backtest ist aber erst ein Anfang. Man muss fragen: Wie viele Trades gab es? Wie hoch war der Drawdown? Hat der Gewinn nur in einer Marktphase stattgefunden? Verändert sich das Ergebnis dramatisch, wenn ein Parameter minimal verschoben wird? Genau diese Folgefragen führen zur Pipeline des Backtesters.

- Backtest: Simulation historischer Trades mit einer bekannten Regel.
- Forward-Test: Teil des Optimierungsfensters, der nicht für die Parameterberechnung genutzt wird, aber später oft in die Auswahl einfließt.
- Out-of-Sample: Daten, die während Optimierung und Auswahl unberührt bleiben.
- Robustheit: Ein Ergebnis bleibt plausibel, wenn Zeitraum, Parameter oder Daten leicht variieren.

Kapitel 2: Curve Fitting, Overfitting und Biases



Curve Fitting zeigt den Unterschied zwischen robustem Signal und historischer Ueberanpassung.

Curve Fitting bedeutet, dass eine Strategie so eng an historische Daten angepasst wird, dass sie nicht mehr das Marktsignal, sondern die Eigenheiten der Stichprobe beschreibt. In der Rueckschau sieht das oft beeindruckend aus: die Equity-Kurve ist glatt, der Profit hoch, der Drawdown klein. Im Live-Handel verschwindet die Magie, weil die Zukunft nicht dieselben Zufallsbewegungen wiederholt. Overfitting ist deshalb kein Randproblem, sondern die zentrale Gefahr jeder Strategieoptimierung.

Je mehr Parameter eine Strategie besitzt, desto groesser wird der Suchraum. Wenn man tausende Kombinationen testet, ist es statistisch fast unvermeidbar, dass einige Kombinationen zufaellig gut aussehen. Das ist kein Betrug, sondern Mathematik: Viele Vergleiche erzeugen viele Chancen auf Scheintreffer. Der Backtester reagiert darauf nicht mit einem einzigen magischen Score, sondern mit einer Abfolge von Filtern, Sensitivitaetsmessungen, KI-Analyse und abschliessender Validierung.

Die Quellenlage bestaetigt diese Vorsicht. QuantStart betont, dass Backtests durch Biases systematisch zu optimistisch wirken koennen. Investopedia beschreibt die Bedeutung von Korrelation zwischen Backtest, Out-of-Sample und Forward/Paper-Test. AlgoTrading101 nennt Look-ahead Bias, Survivorship Bias und Curve Fitting als typische Risiken. Walk-Forward-Ansatz und finale Holdout-Fenster werden in modernen Best-Practice-Texten als Gegenmittel beschrieben, aber nie als Garantie.

In der Praxis ist Curve Fitting nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhaengig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer spaeter nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufaelligen Optimierungsspur.

Fuer das Projekt bedeutet das konkret: ForwardSplit und ValidationResult kodieren im Projekt die Unterscheidung zwischen Auswahlfenster und echter Validierung. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberflaeche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Ein anschauliches Beispiel: Eine Strategie hat einen Take-Profit-Parameter. Bei 48, 49, 50, 51 und 52 Punkten bleibt der Gewinn aehnlich. Das ist ein Plateau und damit ein robuster Hinweis. Wenn aber nur der Wert 50 extrem gut ist und 49 sowie 51 sofort einbrechen, ist der Parameter ein Peak. Peaks koennen auftreten, weil genau diese historische Stichprobe zufaellig passte. Der Backtester versucht, solche Peaks durch Sensitivitaetskurven, CV-Werte und KI-Kurvenformanalyse sichtbar zu machen.

Wichtig ist auch: Ein Forward-Test innerhalb einer MT5-Optimierung ist wertvoll, aber er bleibt Teil der Auswahl. Sobald man die besten Paesse anhand ihrer Forward- Kennzahlen sortiert, filtert oder exportiert, wurde dieses Fenster verbraucht. Es ist dann kein komplett unberuehrter Beweis mehr. Genau darum fuegt das Projekt Schritt 7 hinzu: erst nach Portfolio-Auswahl wird auf einem spaeteren Fenster ein einfacher Backtest mit den finalen Parametern ausgefuehrt.

Kapitel 3: Architektur des Backtester-Projekts



Architekturuebersicht des Backtester-Projekts.

Die Anwendung ist ein Java-17/Maven-Projekt mit JavaFX als aktueller Hauptoberfläche, einer älteren Swing-Schicht, SQLite als lokaler Persistenz und OpenPDF/ReportLab-ähnlicher Report-Logik im Java-Code. Maven baut eine Fat-JAR mit `com.backtester.Main` als Einstiegspunkt. Der Main-Pfad prüft zuerst den CLI-Modus, initialisiert `AppConfig`, räumt alte MetaTrader-Prozesse auf und startet dann die JavaFX-App.

Architektonisch gibt es vier große Ströme. Der Bedien-Strom beginnt in `MainView` und den JavaFX-Views. Der Ausführungs-Strom geht in Engine-Klassen wie `BacktestRunner` und `OptimizationRunner`. Der Ergebnis-Strom läuft über Report-Klassen, die XML/HTM parsen und Scorecards erzeugen. Der Gedächtnis-Strom endet in `DatabaseManager`, der Einstellungen, Historie, Workflows, Sensitivitätsdaten und Reviews in SQLite hält.

Eine wichtige Projektbesonderheit ist die Koexistenz von JavaFX und Swing. Swing ist nicht wertloser Altbestand: viele Konzepte und Panels zeigen die Entwicklung des Werkzeugs und bleiben als Funktionsschicht vorhanden. Für die aktuelle Nutzerführung ist jedoch JavaFX entscheidend, besonders `WorkflowView`, `WorkflowConfigDialogs`, `ControllingView` und die spezialisierten Dialoge für Strategieauswertung.

In der Praxis ist Architektur nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhängig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer später nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufälligen Optimierungsspur.

Für das Projekt bedeutet das konkret: die Pakete trennen UI, Engine, Reporting, Persistenz und Datenimport so, dass jede Schicht eine erkennbare Verantwortung trägt. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberfläche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Fuer Entwickler ist die wichtigste Architekturregel: Die UI darf nicht zur einzigen Wahrheit werden. Wenn ein Button eine Strategie exportiert, muss die fachliche Entscheidung im WorkflowEngine-Zustand, in Validierungsergebnissen und in Datenbankeintraegen wiederzufinden sein. Nur so kann das Projekt spaeter erweitert werden, ohne dass die Anti-Curvefitting-Gates durch eine neue Oberflaechenaktion umgangen werden.

Kapitel 4: Benutzeroberflaeche und Bedienlogik

Die primäre Oberfläche ist JavaFX. JavaFXMain erzeugt eine Szene mit MainView, lädt `antigravity.css` und setzt den Fenstertitel. MainView organisiert die grossen Arbeitsbereiche: Backtest, Multi-Backtest, Optimizer, Robustness, Dukascopy, History, Settings, Help, Workflow und Controlling. Jeder Bereich entspricht einem fachlichen Arbeitsmodus, nicht nur einer visuellen Registerkarte.

BacktestView dient dem Einzeltest. Hier werden Expert Advisor, Symbol, Zeitraum, Modell, Kontoannahmen und Parameterprofil ausgewählt. OptimizationView führt in den MT5-Optimizer, inklusive Parameter-Tabelle, AutoConfig, Forward-Konfiguration und Ergebnisanalyse. MultiBacktestView skaliert Einzeltests über mehrere EAs, Symbole und Perioden. DukascopyView verbindet externe Tickdaten mit dem lokalen MT5-Datenmodell.

WorkflowView ist die didaktisch wichtigste Oberfläche. Sie macht die Pipeline sichtbar: Setup, Optimierung, Diversity-Auswahl, Sensitivität, KI-Bewertung, Portfolio-Auswahl und Step-7-Validierung. Der Nutzer sieht dadurch nicht nur Ergebnisse, sondern auch den Reifegrad der Strategie. Eine Strategie nach Schritt 3 ist ein Kandidat. Eine Strategie nach Schritt 6 ist ein Portfolio-Vorschlag. Eine Strategie nach bestandem Schritt 7 ist deutlich besser abgesichert.



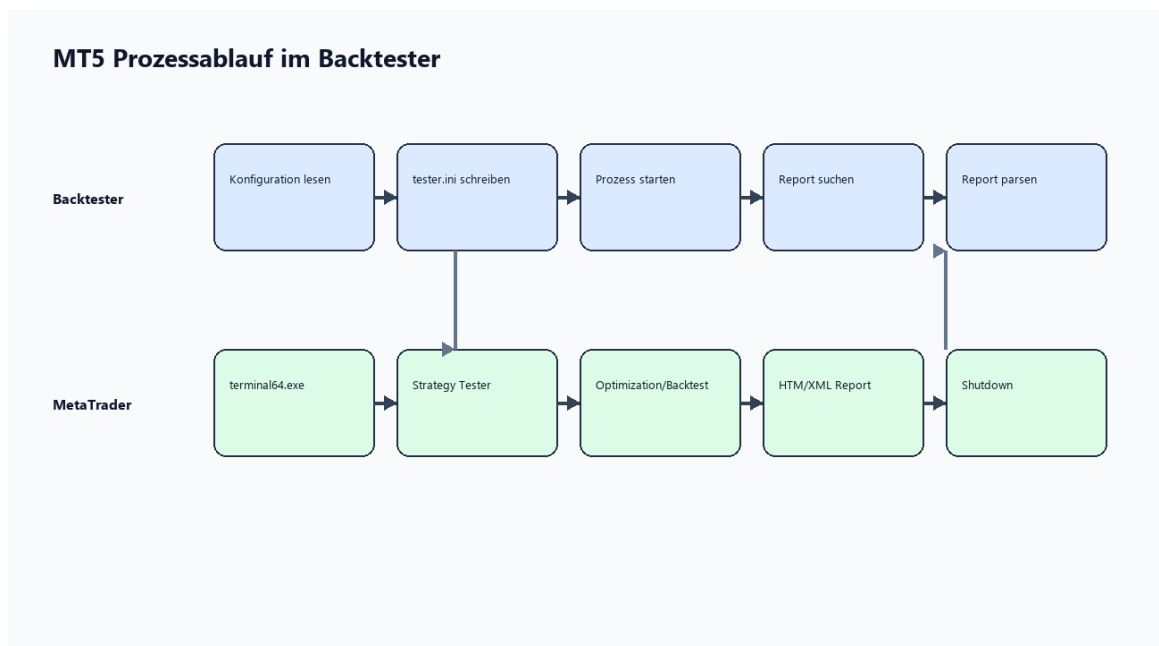
Abbildung: Gesamtplattform des Backtesters.

In der Praxis ist Bedienlogik nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhängig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer später nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufälligen Optimierungsspur.

Für das Projekt bedeutet das konkret: WorkflowView und WorkflowConfigDialogs übersetzen die Engine-Zustände in sichtbare Schritte, Dialoge und Gates. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberfläche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

ControllingView ist die spätere Bewertungs- und Nachtest-Zentrale. Dort werden gespeicherte Strategien betrachtet, manuelle Reviews gepflegt und automatische Nachtests gestartet. Diese Sicht ist wichtig, weil robuste Strategieentwicklung nicht mit einem Export endet. Nachtests, Kommentare und längere Historie machen sichtbar, ob ein Kandidat im Alltag weiter plausibel bleibt.

Kapitel 5: Engine Deep Dive



Vom Java-Klick zum MetaTrader-Report.

Die Engine ist die Arbeitsschicht des Backtesters. BacktestRunner erzeugt einen Ausgabepfad, schreibt tester.ini, bereinigt alte Reports, prüft auf laufende MetaTrader-Prozesse, startet terminal.exe oder terminal64.exe und konsumiert Prozessausgaben, um Deadlocks zu vermeiden. Danach wartet er auf Abschluss oder Timeout, sucht den erzeugten Report und übergibt ihn an ReportParser.

IniGenerator ist klein, aber kritisch. Er kapselt die Unterschiede zwischen MT4 und MT5. MT4 erwartet andere Schlüssel wie TestExpert und TestSymbol, MT5 nutzt Expert und Symbol. Auch Modellwerte und Konfigurationspfade unterscheiden sich. Indem diese Logik zentral bleibt, müssen Runner und UI nicht an jeder Stelle die Plattfordetails kennen.

OptimizationRunner arbeitet ähnlich wie BacktestRunner, jedoch mit Optimierungsparametern, Agentensteuerung, ForwardMode und XML-Parsing. Die Ergebnisobjekte landen in OptimizationResult. Dort werden Backtest-Passes und Forward-Passes zu CombinedPass-Strukturen zusammengeführt. Diese Kombination ist die Grundlage für Ranking, Filterung, Sensitivität und Export.

WorkflowEngine ist der fachliche Kern. Sie hält den Zustand der sieben Pipeline-Schritte, speichert und lädt Strategie-Konfigurationen, erzeugt Optimierungsruns, filtert Kandidaten, startet Sensitivitätsanalysen, ruft die KI-Bewertung auf, kombiniert Performance- und Stabilitätsscore und exportiert die finalen Strategien. Besonders wichtig ist die Regel, dass vorhandene Step-7-Validierungsergebnisse den Best-Ordner-Gate beeinflussen: Nicht bestandene oder nicht eindeutig bestandene Strategien werden nicht stillschweigend als beste Strategien kopiert.

SensitivityRunner variiert Parameter um den optimierten Wert und misst, wie stark Profit und Kennlinien reagieren. String-, Enum- und boolean-artige Parameter werden ausgelassen, weil sie keine sinnvolle kontinuierliche Kennlinie liefern. RobustnessRunner führt ähnliche Gedanken in Form von zeitverschobenen Scans und Plateau-Betrachtungen weiter. ForwardSplit spiegelt die MT5-Forward-Aufteilung, damit BT- und FW-Fenster in der Analyse nicht auseinanderdriften.

In der Praxis ist Engine-Design nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhängig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein

Nutzer spaeter nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufaelligen Optimierungsspur.

Fuer das Projekt bedeutet das konkret: die Runner kapseln Seiteneffekte, waehrend Result-Objekte und WorkflowEngine die fachlichen Entscheidungen tragen. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberflaeche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Kapitel 6: Reporting, Scoring und Persistenz

Persistenzmodell: SQLite als Gedächtnis der Anwendung



SQLite als Gedächtnis der Anwendung.

Reporting ist im Projekt mehr als Formatierung. Reports sind Belege. ReportParser extrahiert Kennzahlen aus MT4/MT5-Reports, OptimizationReportParser liest Optimierungsergebnisse, MultiReportGenerator erzeugt aggregierte HTML-Reports und RobustnessScorecardGenerator visualisiert die Bewertungslogik. PdfReportGenerator erzeugt Strategie-Reports fuer Exportpakete.

Der Score ist bewusst mehrsaeulig. Die aktuelle ScoreWeights-Klasse enthaelt Gewichte fuer Backtest-Profitabilitaet, Forward-Profitabilitaet, Konsistenz, Risiko, Sharpe-basierte Equity-Konsistenz, Stichprobengroesse, Forward-Trades und Recovery. Fruehere synthetische oder hart kodierte Saeulen wurden entfernt. Das ist fachlich sauber, weil ein Score nur so gut ist wie die Daten, aus denen er besteht.

DatabaseManager legt die SQLite-Datenbank im Benutzerprofil unter .mt5_backtester an. Dort liegen HISTORY_RUNS, WORKFLOW_STATE, SENSITIVITY_DETAIL, APP_SETTINGS, KI_REPORTS, MULTI_BACKTEST_BATCHES, EA_PARAMETER_SETTINGS, STRATEGY_REVIEWS, STRATEGY_AUTOMATIC_REVIEWS und weitere Tabellen. Diese Datenbank ist die Bruecke zwischen laufender GUI, Historie, Controlling und MCP-Server.

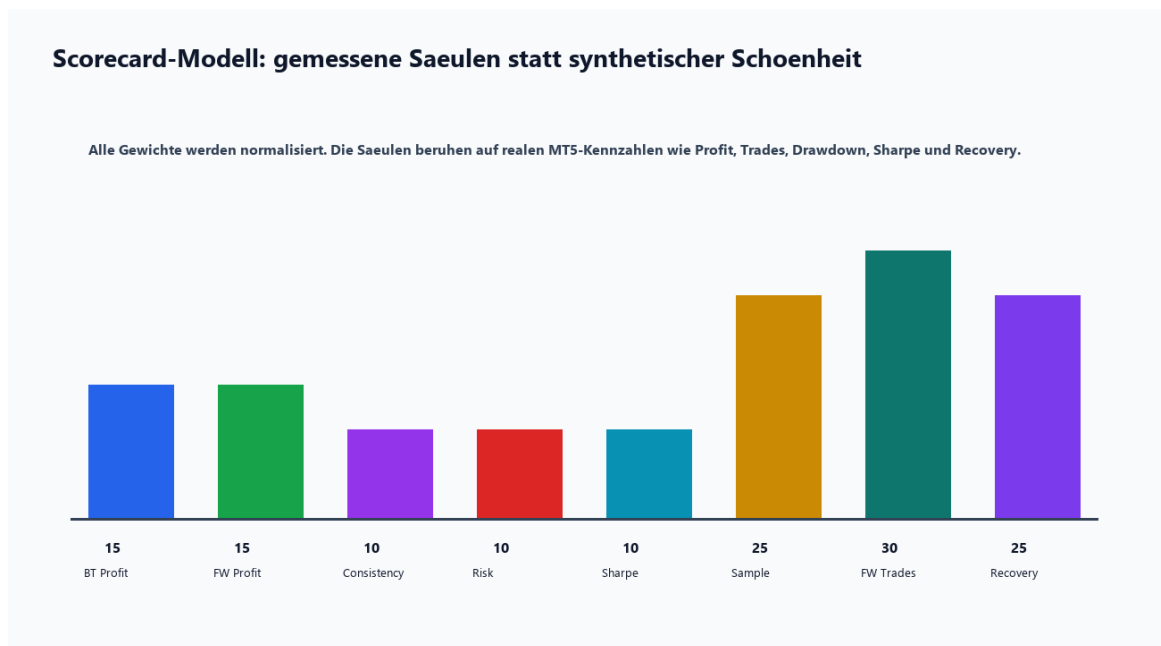
In der Praxis ist Persistenz nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhaengig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer spaeter nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufaelligen Optimierungsspur.

Fuer das Projekt bedeutet das konkret: DatabaseManager macht Ergebnisse wiederherstellbar und verhindert, dass eine Analyse nur als fluechtiger UI-Zustand existiert. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberflaeche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Der MCP-Server in mcp-server/backtester_mcp.py oeffnet die SQLite-Datenbank lesend und stellt Tools wie get_sensitivity_overview, get_sensitivity_for_pass, get_fragile_parameters, get_robust_strategies,

`get_parameter_curve`, `get_optimization_history` und `query_database` bereit. Damit kann ein lokaler KI-Assistent nicht nur freie Texte lesen, sondern strukturierte Backtester-Daten abfragen.

Kapitel 7: Der KI-Ansatz



Scorecard-Modell mit realen Kennzahlen.

Die KI im Backtester ist kein Handelssystem und kein Ersatz fuer Validierung. Sie ist ein Analysewerkzeug fuer Muster, die in Tabellen schwer erkennbar sind. Der LlmAnalysisService laedt Sensitivitaetsdaten aus der Datenbank, fuegt Performance- Kennzahlen hinzu, baut einen deutschen Prompt und ruft OpenRouter auf. Das Modell soll pro Pass eine Tabelle, STABILITY_SCORE-Zeilen und eine knappe Begrueundung liefern.

Der Prompt zwingt die KI zu einer strukturierten Aufgabe: Kurvenformanalyse, CV-Analyse, Performance-Kontext und BT/FW-Konsistenz. Die KI soll unterscheiden, ob ein Parameter ein Plateau, eine Glocke, einen Peak, eine Klippe oder chaotisches Verhalten zeigt. Diese Begriffe sind didaktisch stark, weil sie den Nutzer von reinen Kennzahlen zu Formverstaendnis fuehren.

Im Workflow wird der KI-Score nicht absolut gesetzt. Step 6 kombiniert den numerischen Combined Score mit dem KI-Stabilitaetsscore. Standardmaessig zaehlt Performance zu 60 Prozent und KI-Stabilitaet zu 40 Prozent. Gleichzeitig gibt es ein KI-Gate: Kandidaten mit sehr niedriger KI-Bewertung werden ausgefiltert. Wenn alle Kandidaten durchfallen, erzeugt der Export eine sichtbare Warnung, damit ein Notfall-Fallback nicht wie eine normale Validierung aussieht.

In der Praxis ist KI-Auswertung nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhaengig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer spaeter nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufaelligen Optimierungsspur.

Fuer das Projekt bedeutet das konkret: LlmAnalysisService nutzt die normalisierte Tabelle SENSITIVITY_DETAIL, sodass die KI nicht raten muss, sondern konkrete Kennlinien und Kennzahlen erhaelt. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberflaeche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Grenzen bleiben wichtig. Ein Sprachmodell kann Plausibilität formulieren und Muster benennen, aber es erzeugt keine statistische Gewissheit. Es kennt weder die zukünftige Marktstruktur noch die tatsächliche Broker-Ausführung. Darum darf die KI-Auswertung nie den Step-7-Backtest ersetzen. Sie ist ein sehr nützlicher Filter zwischen Sensitivität und Portfolio-Auswahl, aber der letzte Beleg muss aus Daten kommen, nicht aus Sprache.

Kapitel 8: Datenversorgung mit Dukascopy und MT5 Custom Symbols

Backtests sind nur so gut wie ihre Daten. Das Projekt enthaelt deshalb eine Dukascopy-Schicht. DukascopyDownloader baut stundenweise Download-Aufgaben, speichert BI5-Dateien in einer Symbol/Jahr/Monat/Tag-Struktur und kennt symbolabhangige Preis-Punkt-Multiplikatoren. BI5Decoder liest die LZMA-komprimierten Binardateien und erzeugt Tick-Objekte mit Bid, Ask, Volumen und Zeitstempel.

CsvConverter aggregiert Ticks zu M1-Bars und schreibt CSV-Dateien in einem Format, das fuer den Import in MetaTrader geeignet ist. Mt5DataImporter erzeugt und startet ein MQL5-Skript, um CSV-Daten als Custom Symbol in MT5 zu importieren. CustomSymbolManager merkt sich lokale Symbol-Metadaten wie Originalsymbol, Datenzeitraum, Digits und Aktualisierungsdatum.

Der didaktische Nutzen dieser Schicht ist gross: Sie trennt den Test von einem zufaelligen Brokerfeed. Das macht Ergebnisse nicht automatisch wahr, aber es macht Datenqualitaet bewusster. Ein Nutzer kann sehen, welche Daten geladen wurden, welche Zeitraeume fehlen und welche Symbole als Custom Symbols fuer Tests verfuegbar sind.

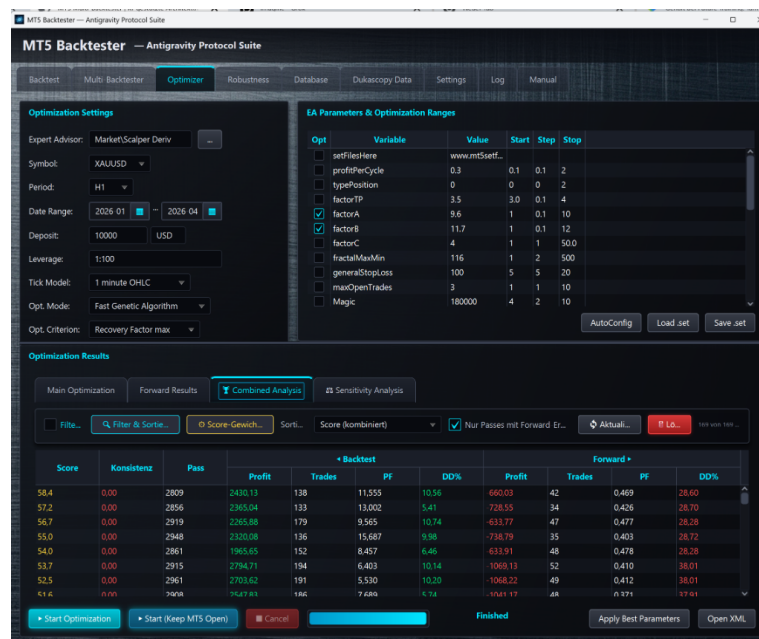


Abbildung: Optimizer-Arbeitsbereich.

In der Praxis ist Datenversorgung nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhaengig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer spaeter nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufaelligen Optimierungsspur.

Fuer das Projekt bedeutet das konkret: DukascopyDownloader, BI5Decoder, CsvConverter und Mt5DataImporter bilden eine Pipeline von externer Tickquelle bis MT5-Testumgebung. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberflaeche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Kapitel 9: Parameter-Referenz

Parameter sind die Sprache, in der der Backtester mit MetaTrader, der Datenbank, der KI und dem Nutzer spricht. Ein Parameter ist dabei nie nur ein Feld. Er hat einen Ort, eine Default-Annahme, einen fachlichen Effekt und oft auch eine Nebenwirkung auf Robustheit oder Reproduzierbarkeit. Dieses Kapitel sammelt die wichtigsten Parametergruppen.

EA-Parameter werden ueber EaParameter und EaParameterManager verwaltet. Ein Parameter kann Name, Anzeigename, Wert, Default-Wert, Sektion, Optimierungsstart, Schrittweite, Ende, Aktivierungsflag und Typinformation tragen. Fuer SET-Dateien ist entscheidend, dass Werte korrekt geschrieben und mit optimierten Passes zusammengefuehrt werden. Bei falscher Zuordnung wuerde ein Report eine andere Strategie beschreiben als die exportierte Datei.

Die Score-Parameter verdienen besondere Vorsicht. Mehr Gewicht fuer Forward-Trades bestraft duenne Evidenz. Mehr Gewicht fuer Recovery belohnt Erholung nach Drawdown. Mehr Gewicht fuer Profit kann Strategien nach oben schieben, die fachlich fragiler sind. Darum sind die Gewichte konfigurierbar, aber sie sollten nicht nachtraeglich so eingestellt werden, dass ein Lieblingskandidat gewinnt.

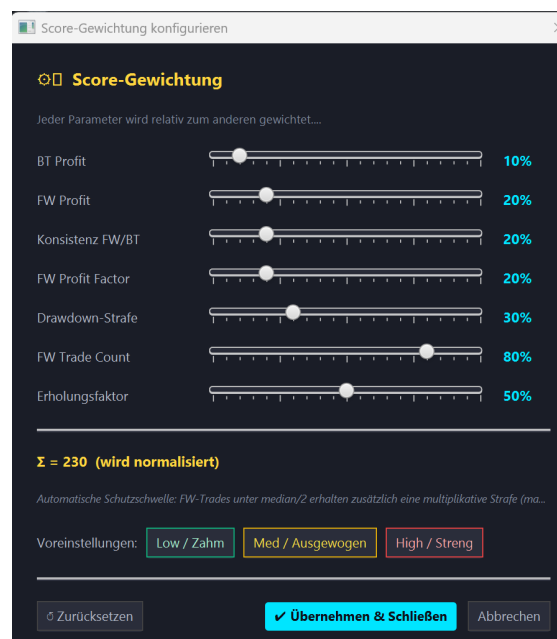


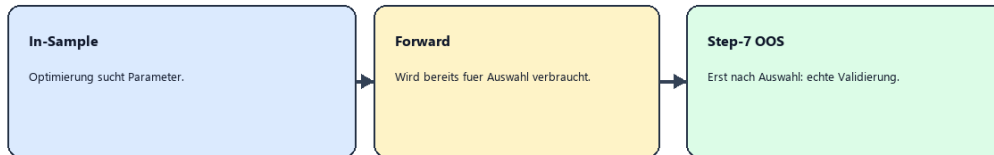
Abbildung: Score-Gewichtung und Ranking.

In der Praxis ist Parameter nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhaengig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer spaeter nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufaelligen Optimierungsspur.

Fuer das Projekt bedeutet das konkret: BacktestConfig, OptimizationConfig, MultiBacktestConfig, WorkflowEngine und ScoreWeights definieren gemeinsam die oeffentliche Fachsprache des Tools. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberflaeche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Kapitel 10: Robuste Optimierung in der Praxis

Warum Step 7 noetig ist



Wenn tausende Paesse getestet werden, findet man fast immer Gewinner im Forward-Fenster. Das ist noch kein Beweis fuer Live-Robustheit.

Step 7 als echte nachgelagerte Out-of-Sample-Validierung.

Eine saubere Pipeline beginnt vor dem ersten Klick. Der Nutzer legt fest, welcher Zeitraum fuer Entwicklung, welcher fuer Forward-Auswahl und welcher spaeter fuer echte Validierung reserviert wird. Wenn das Enddatum der Optimierung bereits heute ist, bleibt kein spaeteres Fenster fuer Step 7. Der Backtester erkennt solche Situationen und verlangt ein brauchbares Validierungsfenster.

Schritt 1 sammelt Strategie, Symbol, Zeitraum, Kontoannahmen und Parameter. Schritt 2 laesst MT5 optimieren. Schritt 3 filtert Kandidaten nicht nur nach Profit, sondern auch nach Forward-Ergebnis, Trades, Drawdown und Diversity. Schritt 4 misst Sensitivitaet, damit einzelne Glueckstreffer sichtbar werden. Schritt 5 laesst die KI Kurvenformen erklaren. Schritt 6 exportiert ein kleines Portfolio. Schritt 7 testet dieses Portfolio auf Daten, die vorher nicht zur Auswahl gehoerten.

In der Praxis sollte ein Nutzer nie nur den hoechsten Score betrachten. Ein Kandidat mit Score 78, breitem Plateau, 80 Forward-Trades und moderatem Drawdown ist oft interessanter als ein Kandidat mit Score 91, aber nur 6 Forward-Trades und einer Peak-Kennlinie. Robustheit bedeutet nicht maximalen historischen Gewinn, sondern die hoechste Chance, dass die beobachtete Kante kein Zufallsprodukt war.

In der Praxis ist Optimierungspraxis nie nur ein einzelner Knopf. Es ist eine Kette von Annahmen: Welche Daten gelten als bekannt, welche Kosten werden simuliert, welche Parameter wurden bereits gesehen und welche Kennzahlen sind wirklich unabhaengig? Der Backtester macht diese Kette sichtbar, weil fast jeder Schritt eine eigene Klasse, einen eigenen Report oder einen eigenen Datenbankeintrag besitzt. Dadurch kann ein Nutzer spaeter nachvollziehen, ob eine Entscheidung aus dem Marktverhalten entstand oder aus einer zufaelligen Optimierungsspur.

Fuer das Projekt bedeutet das konkret: die sieben Workflow-Schritte reduzieren Curve Fitting, indem sie mehrere unabhaengige Fragen stellen statt eine einzige Gewinnzahl zu maximieren. Diese technische Entscheidung hat eine fachliche Wirkung. Sie trennt Bedienkomfort von Bewertungslogik und verhindert, dass die Oberflaeche allein zur Quelle der Wahrheit wird. Das ist wichtig, weil robuste Strategieentwicklung wiederholbar sein muss. Ein gutes Ergebnis ist nur dann ernst zu nehmen, wenn derselbe Ablauf mit denselben Daten und Parametern wiederhergestellt werden kann.

Der wichtigste operative Satz lautet: Kein Best-Ordner ohne ernsthafte Validierung. Wenn Step-7-Ergebnisse existieren, duerfen nur PASSED-Kandidaten in den Best-Ordner. FAILED, NO_TRADES

oder ERROR sind keine Kleinigkeit, sondern eine rote Linie. Ein NO_TRADES-Ergebnis kann bedeuten, dass das Fenster zu kurz war oder die Strategie in dieser Marktphase keine Signale hatte. Auch das ist Information.

Fazit

Der Backtester ist mehr als ein Automatisierer fuer MetaTrader. Er ist ein methodisches Werkzeug, das die gefaehrlichste Versuchung der Strategieentwicklung sichtbar macht: eine schoene Vergangenheit mit einer robusten Zukunft zu verwechseln. Seine Staerke liegt in der Kombination aus Prozessautomatisierung, strukturierter Persistenz, mehrsaeuligem Scoring, Sensitivitaet, KI-gestuetzter Musteranalyse und abschliessender Out-of-Sample-Validierung.

Fuer Nutzer bedeutet das: Das Tool nimmt Arbeit ab, aber nicht Verantwortung. Man muss weiterhin Datenqualitaet, Trade-Anzahl, Drawdown, Marktregime und Plausibilitaet beurteilen. Fuer Entwickler bedeutet es: Jede Erweiterung sollte die Trennung von Optimierung, Auswahl und Validierung respektieren. Der beste Code in diesem Projekt ist nicht der, der den hoechsten Score erzeugt, sondern der, der falsche Sicherheit schwerer macht.

Anhang A: Parameter-Referenz

Die folgende Tabelle fasst die wichtigsten Projektparameter zusammen. Sie ist bewusst breit angelegt: Nutzer erkennen die Wirkung im Tool, Entwickler erkennen die betroffenen Konfigurationsgruppen.

Parameter	Gruppe	Bedeutung	Typische Werte
MT5 Terminal Path	config	Pfad zur terminal64.exe. Ohne gueltigen Pfad kann kein MT5-Prozess gestartet werden.	C:\Program Files\MetaTrader 5\terminal64.exe
MT4 Terminal Path	config	Pfad zur terminal.exe. Wird genutzt, wenn ein Expert Advisor als MT4-Artefakt erkannt wird.	C:\Program Files\MetaTrader 4\terminal.exe
Data Directory	config	Ablage fuer Dukascopy-Daten, konvertierte CSVs und lokale Marktdaten.	data
Reports Directory	config	Ziel fuer Backtest-, Optimierungs- und HTML/PDF-Reports.	backtest_reports
Export Directory	config	Ziel fuer normale Portfolio- und SET-Datei-Exporte.	exports
Best Export Directory	config	Ziel fuer sehr gute Strategien; nach Step 7 nur fuer PASSED-Validierungen.	exports_gut
Portable Mode	config	Startet MT5 mit /portable, damit Pfade und Profile kontrollierbarer bleiben.	true/false
Backtest Timeout	config	Freeze-Schutz fuer MetaTrader-Prozesse. Lange Optimierungen brauchen hoeheren Wert.	Minuten
Broker Timezone Offset	config	Zeitverschiebung beim Umwandeln externer Daten in Brokerzeit.	0
Expert	backtest	Pfad oder relativer Name des Expert Advisors, der getestet wird.	MQL5\Experts\EA.ex5
ExpertParameters	backtest	SET-Datei oder Parameterprofil, das MT5 laden soll.	*.set
Symbol	backtest	Markt, auf dem getestet wird. Muss in MT5 vorhanden sein.	EURUSD, XAUUSD
Period	backtest	Zeiteinheit des Tests.	M1, M5, M15, H1, D1
Model	backtest	MT5-Modell fuer Tick-Qualitaet. Hoehere Genauigkeit kostet Laufzeit.	0, 1, 2
ExecutionMode	backtest	MetaTrader-Ausfuhrungsmodell fuer Order-Simulation.	0
FromDate/ToDate	backtest	Historisches Testfenster. Fachlich entscheidend fuer In-Sample und OOS.	YYYY-MM-DD
Deposit	backtest	Startkapital fuer Performance-Kennzahlen.	10000
Currency	backtest	Kontowaehrung fuer Reports.	USD
Leverage	backtest	Hebelannahme fuer Strategie-Tester.	1:100
ShutdownTerminal	backtest	Soll MT5 nach Abschluss automatisch schliessen.	true
UseVirtualDesktop	backtest	Startet MetaTrader auf Desktop 2, um den Nutzerarbeitsplatz frei zu halten.	true/false
AutoKillMt5	backtest	Erlaubt automatisches Beenden alter MetaTrader-Prozesse.	true/false
VisualMode	backtest	Startet den visuellen Tester fuer manuelle Beobachtung.	false
OptimizationMode	optimizer	0 deaktiviert Optimierung, 1 Complete, 2 Genetic. Genetic ist schneller, Complete gruendlicher.	2
OptimizationCriterion	optimizer	MT5-Kriterium fuer Ranking, etwa Balance, Profit Factor, Recovery oder Sharpe.	0-6
ForwardMode	optimizer	Teilt Optimierungsfenster in Backtest und Forward auf.	0, 1, 2, 3, 4
ForwardDate	optimizer	Custom-Start des Forward-Fensters bei ForwardMode 4.	YYYY-MM-DD
UseLocal/Remote/Cloud	optimizer	Steuert, welche MT5-Agenten fuer Optimierung genutzt werden.	1/0
minBtProfit	workflow	Mindestgewinn im Backtest fuer Step-3-Kandidaten.	0.01
minFwProfit	workflow	Mindestgewinn im Forward-Fenster.	0.01
minBtTrades	workflow	Mindestanzahl Trades im Backtest gegen statistisch duenne Ergebnisse.	100
minFwTrades	workflow	Mindestanzahl Trades im Forward-Fenster.	15
maxBtDd/maxFwDd	workflow	Maximal tolerierter Drawdown in Backtest und Forward.	100

Parameter	Gruppe	Bedeutung	Typische Werte
paramDiffPct	workflow	Diversity-Schwelle fuer Parameterunterschiede zwischen Kandidaten.	0.10
tradeDiffPct	workflow	Diversity-Schwelle fuer abweichende Trade-Anzahlen.	0.15
minDifferentParams	workflow	Mindestzahl unterschiedlicher Parameter fuer Portfolio-Diversitaet.	2
maxStrategiesToSelect	workflow	Maximale Zahl der Kandidaten nach Diversity-Filter.	5
OpenRouter API Key	ki	Lokaler API-Schlüssel fuer LLM-Auswertung; wird in SQLite gespeichert, nicht im Git.	leer
OpenRouter Model	ki	LLM-Modell fuer Stabilitaetsanalyse.	openai/gpt-4o-mini
OpenRouter Prompt	ki	Prompt mit Tabellenformat, Kurvenform-Analyse und Score-Regeln.	DEFAULT_PROMPT
Performance Weight	ki	Gewicht des numerischen Performance-Scores im finalen Ranking.	0.6
Stability Weight	ki	Gewicht des KI-Stabilitaetscores im finalen Ranking.	0.4
wBtProfit	score	Gewicht fuer Backtest-Profitabilitaet.	15
wFwProfit	score	Gewicht fuer Forward-Profitabilitaet.	15
wConsistency	score	Gewicht fuer Verhaeltnis von Forward zu Backtest.	10
wRisk	score	Gewicht fuer Risiko/Drawdown-Verhaeltnis.	10
wEquityConsist	score	Gewicht fuer echte Sharpe-basierte Equity-Konsistenz.	10
wSampleSize	score	Gewicht fuer Stichprobengroesse und Testdauer.	25
wFwTrades	score	Gewicht fuer Anzahl der Forward-Trades.	30
wRecovery	score	Gewicht fuer Recovery Factor.	25
recoveryMin/recoveryMax	score	Skalierungsbereich fuer Recovery-Faktor-Bewertung.	1.0 / 5.0
validationFromDate	validation	Start des echten Step-7-OOS-Fensters; leer bedeutet toDate + 1 Tag.	null
validationToDate	validation	Ende des Step-7-OOS-Fensters; leer bedeutet aktuelles Datum.	null

Anhang B: Sourcecode-Modulindex

Die Hauptanwendung umfasst 79 Java-Dateien in 11 Paketen. Die Tabelle zeigt die aus dem Repository abgeleitete Struktur.

Paket	Dateien	Zeilen	Rolle
com.backtester	1	177	Startpunkt der Anwendung. Main entscheidet zwischen CLI und Desktop-UI, initialisiert die Konfiguration und raumt alte MetaTrader-Prozesse auf.
com.backtester.cli	1	373	Headless Batch-Betrieb fuer automatisierte Laeufe ohne GUI. Relevant fuer reproduzierbare Serien und spaetere Automatisierung.
com.backtester.config	6	1453	Zentrale Projektkonfiguration, Plattform-Erkennung, EA-Parameter, SET-Dateien, Presets und Pfade zu MT4/MT5.
com.backtester.database	3	1463	SQLite-Persistenz fuer Historie, Workflow-State, Sensitivitaetsdaten, KI-Berichte, Reviews und Einstellungen.
com.backtester.dukascopy	3	677	Download, Dekodierung und Umwandlung von Dukascopy-BI5-Tickdaten in nutzbare M1/CSV-Daten.
com.backtester.engine	16	5771	Ausfuehrungsschicht: Backtests, Optimierungen, Sensitivitaet, Robustheit, Workflow-Orchestrierung, Prozessschutz und Forward-Split.
com.backtester.mt5	2	452	Import von CSV-Daten in MT5 Custom Symbols und Verwaltung lokaler Symbol-Metadaten.
com.backtester.report	11	5050	Parser, Ergebnisobjekte, Scorecard, HTML/PDF-Reports, Multi-Reports und Validierungsergebnisse.
com.backtester.tools	2	824	Headless Auswertungs- und Export-Werkzeuge fuer strategische Portfolio-Auswahl und Report-Erzeugung.
com.backtester.ui	14	7271	Aeltere Swing-Oberflaeche mit Panels fuer Backtest, Optimizer, Multi-Backtest, Historie, Dukascopy und Settings.
com.backtester.ui.javaafx	20	20496	Aktuelle primaere JavaFX-Oberflaeche: MainView, WorkflowView, Dialoge, Controlling und moderne Interaktionsschicht.

Klassenubersicht

Paket	Klasse	Zeilen
com.backtester.cli	CliRunner	373
com.backtester.config	AppConfig	317
com.backtester.config	EaParameter	103
com.backtester.config	EaParameterManager	788
com.backtester.config	MetaTraderPlatform	112
com.backtester.config	Preset	44
com.backtester.config	PresetManager	89
com.backtester.database	DatabaseManager	1374
com.backtester.database	EaDbConfig	49
com.backtester.database	HistoryRun	40
com.backtester.dukascopy	Bi5Decoder	151
com.backtester.dukascopy	CsvConverter	203
com.backtester.dukascopy	DukascopyDownloader	323
com.backtester.engine	BacktestConfig	170
com.backtester.engine	BacktestRunner	719
com.backtester.engine	ForwardSplit	73
com.backtester.engine	IniGenerator	204
com.backtester.engine	KiReport	49
com.backtester.engine	LlmAnalysisService	522
com.backtester.engine	Mt5LogTailor	396

Paket	Klasse	Zeilen
com.backtester.engine	Mt5ProcessGuard	138
com.backtester.engine	MultiBacktestConfig	116
com.backtester.engine	MultiBacktestRunner	148
com.backtester.engine	OptimizationConfig	165
com.backtester.engine	OptimizationRunner	390
com.backtester.engine	RobustnessRunner	265
com.backtester.engine	SensitivityRunner	463
com.backtester.engine	VirtualDesktopHelper	282
com.backtester.engine	WorkflowEngine	1671
com.backtester	Main	177
com.backtester.mt5	CustomSymbolManager	133
com.backtester.mt5	Mt5DataImporter	319
com.backtester.report	BacktestResult	163
com.backtester.report	MultiReportGenerator	848
com.backtester.report	OptimizationReportParser	245
com.backtester.report	OptimizationResult	565
com.backtester.report	PdfReportGenerator	902
com.backtester.report	ReportParser	524
com.backtester.report	RobustnessHtmlGenerator	355
com.backtester.report	RobustnessResult	49
com.backtester.report	RobustnessScorecardGenerator	1131
com.backtester.report	SensitivityResult	147
com.backtester.report	ValidationResult	121
com.backtester.tools	DumpOptState	100
com.backtester.tools	StrategyExporter	724
com.backtester.ui	BacktestPanel	786
com.backtester.ui	DbConfigSelectionDialog	242
com.backtester.ui	DukascopyPanel	614
com.backtester.ui	EaConfigDialog	641
com.backtester.ui	EquityChartPanel	314
com.backtester.ui	HelpPanel	180
com.backtester.ui	HistoryPanel	261
com.backtester.ui.javaafx	AppLauncher	9
com.backtester.ui.javaafx	BacktestView	968
com.backtester.ui.javaafx	ControllingView	2275
com.backtester.ui.javaafx	DistributionAnalysisDialog	518
com.backtester.ui.javaafx	DocHelper	793
com.backtester.ui.javaafx	DukascopyView	181
com.backtester.ui.javaafx	EnumAwareParamCell	158
com.backtester.ui.javaafx	HelpView	83
com.backtester.ui.javaafx	HistoryView	384
com.backtester.ui.javaafx	JavaFXMain	35
com.backtester.ui.javaafx	LogView	139
com.backtester.ui.javaafx	MainView	128
com.backtester.ui.javaafx	MultiBacktestView	1462
com.backtester.ui.javaafx	OptimizationStatsDialog	258
com.backtester.ui.javaafx	OptimizationView	4390

Paket	Klasse	Zeilen
com.backtester.ui.javaafx	RobustnessView	1163
com.backtester.ui.javaafx	SettingsView	503
com.backtester.ui.javaafx	StrategyEvaluatorDialog	1717
com.backtester.ui.javaafx	WorkflowConfigDialogs	2448
com.backtester.ui.javaafx	WorkflowView	2884
com.backtester.ui	LogPanel	154
com.backtester.ui	MainFrame	276
com.backtester.ui	MultiBacktestPanel	705
com.backtester.ui	OptimizationPanel	1208
com.backtester.ui	ReportViewerDialog	569
com.backtester.ui	RobustnessPanel	949
com.backtester.ui	SettingsPanel	372

Test- und Qualitätsindex

Die Tests sind keine vollständige formale Spezifikation, aber sie zeigen, welche Verhaltensweisen als schützenswert betrachtet werden: Forward-Split, Workflow-Gates, Scorecard, Persistenz, Parser, Dukascopy und UI-Komponenten.

Testdatei	Zeilen
src\test\java\com\backtester\config\AppConfigTest.java	156
src\test\java\com\backtester\config\EaParameterManagerParameterizedTest.java	46
src\test\java\com\backtester\config\EaParameterManagerTest.java	479
src\test\java\com\backtester\config\EaParameterTest.java	88
src\test\java\com\backtester\database\DatabaseManagerPersistenceTest.java	503
src\test\java\com\backtester\database\EaDbConfigTest.java	35
src\test\java\com\backtester\database\HistoryRunTest.java	37
src\test\java\com\backtester\dukascopy\Bi5DecoderTest.java	112
src\test\java\com\backtester\dukascopy\CsvConverterTest.java	115
src\test\java\com\backtester\dukascopy\DukascopyDownloaderTest.java	115
src\test\java\com\backtester\engine\BacktestConfigTest.java	91
src\test\java\com\backtester\engine\ForwardSplitTest.java	123
src\test\java\com\backtester\engine\IniGeneratorTest.java	102
src\test\java\com\backtester\engine\KiReportTest.java	60
src\test\java\com\backtester\engine\Mt5LogTailerTest.java	74
src\test\java\com\backtester\engine\MultiBacktestConfigTest.java	64
src\test\java\com\backtester\engine\OptimizationConfigTest.java	47
src\test\java\com\backtester\engine\VirtualDesktopHelperTest.java	80
src\test\java\com\backtester\engine\WorkflowDiversityTest.java	323
src\test\java\com\backtester\engine\WorkflowEngineTest.java	792
src\test\java\com\backtester\engine\WorkflowValidationAndGateTest.java	476
src\test\java\com\backtester\mt5\CustomSymbolManagerTest.java	117
src\test\java\com\backtester\report\BacktestPersistenceTest.java	231
src\test\java\com\backtester\report\BacktestResultTest.java	92
src\test\java\com\backtester\report\MassiveCoverageTest.java	1031
src\test\java\com\backtester\report\Mt4AndMt5CompatibilityTest.java	108
src\test\java\com\backtester\report\MultiBatchPersistenceTest.java	252
src\test\java\com\backtester\report\MultiReportGeneratorTest.java	263
src\test\java\com\backtester\report\OptimizationPersistenceTest.java	204
src\test\java\com\backtester\report\OptimizationReportParserTest.java	56
src\test\java\com\backtester\report\OptimizationResultMoreTest.java	259
src\test\java\com\backtester\report\OptimizationResultTest.java	227
src\test\java\com\backtester\report\PdfReportGeneratorTest.java	346
src\test\java\com\backtester\report\ReportParserTest.java	355
src\test\java\com\backtester\report\RobustnessPersistenceTest.java	170
src\test\java\com\backtester\report\RobustnessResultTest.java	78
src\test\java\com\backtester\report\RobustnessScorecardGeneratorTest.java	58
src\test\java\com\backtester\report\ScorecardTestApp.java	99
src\test\java\com\backtester\report\SensitivityResultTest.java	128
src\test\java\com\backtester\report\StrategyEvaluatorTest.java	126
src\test\java\com\backtester\ui\javafx\UiComponentsCoverageTest.java	247

Anhang C: Kritische Klassen und Entwicklerleitfaden

Die folgenden Klassen sind die wichtigsten Orientierungspunkte fuer Entwickler. Sie sind nicht alle gleich gross, aber sie tragen die fachlichen Grenzen des Systems: Prozessstart, Parameterwahrheit, Score, Persistenz, Validierung und Export.

Klasse / Datei	Rolle im Projekt
Main	Startet CLI oder JavaFX, initialisiert AppConfig und entfernt alte MetaTrader-Prozesse.
JavaFXMain	Erzeugt Stage, Scene, CSS und Icon der modernen Benutzeroberflaeche.
MainView	Hauptcontainer fuer Tabs und Views der JavaFX-Anwendung.
WorkflowView	Visualisiert die siebenstufige Pipeline und bindet UI an WorkflowEngine.
WorkflowConfigDialogs	Konfigurationsdialoge fuer Workflow-Schritte, Score-Gewichte und KI-Setup.
ControllingView	Analyse, Review und Nachtest-Zentrale fuer gespeicherte Strategien.
BacktestView	JavaFX-Einzeltestoberflaeche.
OptimizationView	JavaFX-Oberflaeche fuer Optimierungen, Ergebnisanalyse und Sensitivitaet.
DukascopyView	UI fuer Download, Scan, CSV-Konvertierung und MT5-Import von Daten.
AppConfig	Zentrale Pfade, Defaults, Plattform-Erkennung und Verzeichnisse.
MetaTraderPlatform	Abstraktion fuer MT4/MT5-Unterschiede wie Executable, Logs und Report-Endung.
EaParameter	Datenmodell fuer EA-Parameter inklusive Optimierungsbereich.
EaParameterManager	Liest, schreibt, merged und generiert SET-Dateien und Parameterprofile.
BacktestConfig	Konfiguration fuer einzelne Backtests.
OptimizationConfig	Konfiguration fuer MT5-Optimierungen inklusive ForwardMode und Agenten.
MultiBacktestConfig	Erzeugt Einzelkonfigurationen fuer Batch-Kombinationen.
IniGenerator	Schreibt MT4/MT5-kompatible tester.ini-Dateien.
BacktestRunner	Steuert Einzeltestprozess und Reportuebernahme.
OptimizationRunner	Steuert Optimierungsprozess und Optimierungsreport.
WorkflowEngine	State Machine, Persistenz, Gates, Export und Step-7-Validierung.
ForwardSplit	Spiegelt MT5-Forward-Fenster fuer korrekte Sensitivitaetsperioden.
LlmAnalysisService	Baut Prompt aus Sensitivitaetsdaten und ruft OpenRouter auf.
SensitivityRunner	Variiert Parameter und misst CV/Kennlinien fuer BT und FW.
RobustnessRunner	Fuehrt Robustheitsscans mit Parameter- und Zeitverschiebungen aus.
Mt5ProcessGuard	Schuetzt vor stale MetaTrader-Instanzen.
VirtualDesktopHelper	Startet MetaTrader normal oder auf Desktop 2.
ReportParser	Parst Einzeltestreports in BacktestResult.
OptimizationReportParser	Parst Optimierungsreports in OptimizationResult.
OptimizationResult	Haelt Passes, Forward-Passes, CombinedPass und ScoreWeights.
RobustnessScorecardGenerator	Erzeugt HTML-Scorecards und berechnet Overall Score.
PdfReportGenerator	Erzeugt Strategie-PDFs fuer Exportpakete.
ValidationResult	Verdict-Regeln fuer Step-7-OOS-Ergebnisse.
DatabaseManager	Erzeugt Tabellen, speichert History, Settings, Workflow und Reviews.
DukascopyDownloader	Laedt BI5-Dateien stundenweise und verwaltet Fortschritt.
Bi5Decoder	Dekodiert LZMA-komprimierte BI5-Ticks.
CsvConverter	Aggregiert Ticks zu M1-Bars und schreibt CSV.
Mt5DataImporter	Startet MT5 mit Importskript fuer Custom Symbols.
CustomSymbolManager	Speichert Metadaten importierter Symbole.
StrategyExporter	Headless Export- und Controlling-Report-Service.
backtester_mcp.py	MCP-Server fuer lesenden Zugriff auf SQLite-Backtester-Daten.

Workflow-Schrittmatrix

Die Matrix verbindet Benutzeraktion, Code-Ort und fachlichen Kontrollpunkt. Genau diese Verbindung macht das Buch zu einer Projektdokumentation und nicht nur zu einer Bedienungsanleitung.

Schritt	Code-Ort	Aufgabe	Kritischer Punkt
1 Setup	WorkflowEngine.runStep1	Speichert Strategiegrunddaten, EA-Parameter, Zeitraum und Kontoannahmen.	Falsche Zeitraumwahl verhindert spätere OOS-Validierung.
2 MT5-Optimierung	WorkflowEngine.runStep2 / OptimizationRunner	Startet MT5-Optimierung mit ForwardMode und schreibt OptimizationResult.	Zu grosse Suchräume erhöhen Multiple-Testing-Bias.
3 Diversity-Auswahl	WorkflowEngine.runStep3	Filtert CombinedPasses nach Profit, Trades, Drawdown, Score und Diversität.	Ähnliche Pässe dürfen nicht als echtes Portfolio missverstanden werden.
4 Sensitivität	WorkflowEngine.runStep4 / SensitivityRunner	Variert Parameter einzeln und speichert CV sowie Kurven in SENSITIVITY_DETAIL.	Peak-Parameter werden sichtbar; String/Enum-Parameter werden ausgelassen.
5 KI-Bewertung	WorkflowEngine.runStep5 / LlmAnalysisService	Sendet Kennlinien und Performance-Kontext an OpenRouter und parst Stabilitätsscores.	KI bewertet Muster, ersetzt aber keine Datenvalidierung.
6 Portfolio	WorkflowEngine.runStep6 / exportPortfolio	Kombiniert Performance- und KI-Score, exportiert SET/PDF und markiert KI-Gate-Bypass.	Export ist noch keine finale Live-Freigabe.
7 Validierung	WorkflowEngine.runStep7 / ValidationResult	Testet finale Pässe auf nachgelagertem OOS-Fenster und schreibt Validierungsreport.	Nur PASSED darf nach vorhandener Validierung in den Best-Ordner.

Architekturentscheidungen

Die folgenden Entscheidungen sind im Sourcecode sichtbar und sollten bei Erweiterungen respektiert werden. Sie beschreiben nicht nur, wie das Projekt gebaut ist, sondern warum bestimmte Grenzen existieren.

Entscheidung	Umsetzung	Wirkung
MetaTrader bleibt Simulationsmotor	Die Java-App orchestriert, aber ersetzt den MT5/MT4 Strategy Tester nicht.	EA-Verhalten bleibt nah an der Zielplattform.
tester.ini statt GUI-Automation	Konfiguration wird als Datei erzeugt und per CLI gestartet.	Weniger fehleranfällig als Klick-Automation.
Runner kapseln Seiteneffekte	Prozessstart, Logs, Timeouts und Reportdateien liegen in engine.	UI bleibt testbarer und fachlich schlanker.
SQLite im Benutzerprofil	history.db liegt unter .mt5_backtester.	Nutzerdaten werden nicht ins Git geschrieben.
ScoreWeights als Single Source	Ranking und Scorecard lesen dieselben Defaults.	Keine divergierenden Bewertungen zwischen UI und Report.
ForwardSplit isoliert	MT5-Splitlogik ist in eigener Klasse und getestet.	BT/FW-Sensitivität bleibt fachlich korrekt.
Step 7 nach Exportauswahl	Finale OOS-Validierung passiert nach Optimierung und Portfolio-Auswahl.	Forward-Fenster wird nicht als unberührter Beweis missbraucht.
KI als Analyst	LLM interpretiert Kennlinien und Scores, trifft aber nicht allein die Freigabe.	Sprachliche Plausibilität bleibt von Datenvalidierung getrennt.
Best-Ordner-Gate	Validierungsergebnisse beeinflussen Export in exports_gut.	Fehlgeschlagene Kandidaten werden nicht als Top-Strategien präsentiert.
MCP read-only	Der MCP-Server erlaubt nur lesende SQLite-Abfragen.	KI-Assistenten können analysieren, aber Daten nicht verändern.
Dukascopy als Datenpipeline	Bi5-Download, Decode, CSV und MT5-Import sind eigene Schritte.	Datenqualität wird nachvollziehbar.
JavaFX als primäre UI	Moderne Views tragen die aktive Nutzerführung.	Swing kann koexistieren, ohne die Hauptarchitektur zu blockieren.
Reports als Belege	PDF/HTML/SET werden zusammen exportiert.	Strategieentscheidung bleibt nachvollziehbar.
Tests als Methodenschutz	ForwardSplit, Workflow-Gates und Scorecard sind regressionsrelevant.	Fachliche Schutzlogik wird bei Änderungen nicht still gebrochen.

Datenflüsse

Fluss	Kette
Einzeltest	BacktestView -> BacktestConfig -> IniGenerator -> BacktestRunner -> MetaTrader -> ReportParser -> BacktestResult -> DatabaseManager
Optimierung	OptimizationView -> OptimizationConfig -> IniGenerator -> OptimizationRunner -> MT5 XML -> OptimizationReportParser -> OptimizationResult
Workflow	WorkflowView -> WorkflowEngine -> OptimizationRunner/SensitivityRunner/LlmAnalysisService/BacktestRunner -> ValidationResult -> Export
Sensitivität	CombinedPass -> SensitivityRunner -> Parameter-Sweep -> SENSITIVITY_DETAIL -> RobustnessScorecard/KI
Dukascopy	DukascopyView -> DukascopyDownloader -> Bi5 -> Bi5Decoder -> CsvConverter -> Mt5DataImporter -> Custom Symbol
Controlling	DatabaseManager -> History/Reviews/AutomaticReviews -> ControllingView -> Nachtest/Export
MCP	Claude oder anderer Client -> backtester_mcp.py -> read-only SQLite -> JSON-Antwort

Entwicklerleitfaden: Erweiterungen ohne Methodikbruch

Neue Funktionen sollten zuerst entscheiden, welche Schicht betroffen ist: UI, Engine, Report, Persistenz oder Datenimport. Ein neues UI-Element darf keine fachliche Regel umgehen, die bereits in WorkflowEngine, ScoreWeights oder ValidationResult kodiert ist.

Neue Score-Komponenten muessen auf real gemessenen Daten beruhen. Historisch wurden synthetische Saeulen entfernt, weil sie ein Gefuehl von Genauigkeit erzeugen konnten, ohne echte MetaTrader-Messwerte zu nutzen.

Neue Datenbankfelder brauchen defensive Migrationen. DatabaseManager wird bei bestehenden Nutzern laufen, deren history.db bereits alte Tabellen besitzt. Migrationen muessen fehlende Spalten erkennen und alte Daten erhalten.

Neue Runner sollten Seiteneffekte kapseln. Prozessesstart, Dateikopien, Timeouts und Logtailing gehoeren in die Engine. UI-Klassen sollen den Start ausloesen und Status anzeigen, aber nicht selbst MetaTrader-Prozessdetails duplizieren.

Neue KI-Funktionen muessen zwischen Analyse und Entscheidung unterscheiden. Ein LLM kann Kennlinien erklaren, aber eine PASSED-Validierung in Step 7 darf nicht durch einen sprachlichen Score ersetzt werden.

Neue Exportwege muessen die gleiche Gate-Logik respektieren wie exportPortfolio: Wenn Validierungsergebnisse existieren, sind nur PASSED-Kandidaten fuer den Best-Ordner geeignet.

Neue Tests sollten die fachliche Grenze abdecken, nicht nur die Codezeile. Besonders wichtig sind ForwardSplit, WorkflowValidationAndGateTest, Scorecard-Tests, Parser-Tests und Persistenztests.

Bei Refactorings der UI ist darauf zu achten, dass JavaFX die primaere Oberflaeche ist, Swing aber weiterhin relevante Bedien- und Dokumentationsspuren besitzt. Entfernen ohne Migrationsplan wuerde Nutzerpfade brechen.

Anhang D: Betriebschecklisten und Troubleshooting

Diese Checklisten sind als Arbeitsblatt gedacht. Sie helfen, eine Strategie nicht zu frueh als robust einzustufen und typische Betriebsprobleme schneller zu diagnostizieren.

Phase	Pruefung	Warum wichtig
Vor Optimierung	Zeitfenster festlegen	In-Sample, Forward und spaeteres OOS-Fenster bewusst trennen.
Vor Optimierung	Datenqualitaet pruefen	Symbolhistorie, Custom Symbols und fehlende Tage kontrollieren.
Vor Optimierung	Parameterzahl reduzieren	Nur fachlich sinnvolle Parameter optimieren, nicht jedes Feld.
Vor Optimierung	Kostenannahmen pruefen	Spread, Kommission und Ausfuehrungsmodell im MT5-Kontext verstehen.
Step 2	ForwardMode aktivieren	Ohne Forward fehlt eine wichtige Auswahlperspektive.
Step 2	Optimierungsmodus waehlen	Genetic fuer Suche, Complete fuer kleinere, kritische Raeume.
Step 3	Mindesttrades nutzen	Kleine Stichproben nicht mit robusten Strategien verwechseln.
Step 3	Drawdown begrenzen	Profit ohne Risiko-Kontext ist gefaehrlich.
Step 3	Diversity erzwingen	Nicht fuer fast identische Paesse exportieren.
Step 4	CV-Werte lesen	Worst-CV ist wichtiger als ein schoener Durchschnitt.
Step 4	Kurvenformen pruefen	Plateaus sind besser als Peaks.
Step 5	KI-Bericht nicht blind glauben	KI erklart Muster, ersetzt aber keine Validierung.
Step 6	KI-Gate-Warnungen ernst nehmen	Ein Bypass ist kein Qualitaetssiegel.
Step 7	Validierungsfenster sauber waehlen	Fenster muss nach Auswahl liegen und genug Marktaktivitaet enthalten.
Step 7	NO_TRADES analysieren	Kein Trade ist keine bestandene Robustheit, sondern fehlende Evidenz.
Nach Export	Best-Ordner pruefen	Nur PASSED-Strategien sollen dort landen, wenn Validierungen existieren.
Nach Export	SET und PDF zusammen halten	Parameterdatei und Report muessen dieselbe Pass-Identitaet tragen.
Betrieb	History pflegen	Alte Laeufe nicht loeschen, bevor wichtige Vergleiche gesichert sind.
Betrieb	Reviews schreiben	Manuelle Beobachtungen im Controlling dokumentieren.
Entwicklung	ForwardSplit-Test beachten	Aenderungen am Split koennen die Anti-Curvefitting-Aussage zerstoenen.
Entwicklung	ScoreWeights zentral halten	Keine parallelen Defaults in UI oder Reports einfuehren.
Entwicklung	DB-Migrationen defensiv bauen	Bestehende Nutzer-Daten duerfen nicht brechen.
Entwicklung	Runner-Seiteneffekte kapseln	Prozessstart, Reportdateien und Timeouts gehoeren in die Engine.
Entwicklung	Tests erweitern	Neue Gates oder Parameter brauchen Regressionstests.

Troubleshooting

Symptom	Diagnose / Loesung
MetaTrader startet und beendet sich sofort	Oft laeuft bereits eine Instanz im gleichen portable-Verzeichnis. ProcessGuard/AutoKill pruefen und MT5 sauber beenden.
Kein Report wird gefunden	Report-Pfad, ShutdownTerminal, alte Reportdateien und Tester-Logs pruefen. BacktestRunner wartet auf erwartete Reportnamen.
Optimierung hat 0 Paesse	Parameterbereiche, OptimizationMode, EA-Kompilierung und MT5-Tester-Log pruefen.
Forward-Werte fehlen	ForwardMode deaktiviert oder MT5 hat keinen Forward-Report erzeugt. requireForward-Filter beachten.
Step 7 ist nicht startbar	Validierungsfenster muss nach dem Optimierungs-ToDate liegen und ein sinnvolles Enddatum besitzen.
Strategie wird nicht in Best kopiert	Nach vorhandenen Step-7-Ergebnissen duerfen nur PASSED-Kandidaten in den Best-Ordner.
KI-Analyse meldet keinen API-Key	OpenRouter-Schluessel in KI-Einstellungen setzen; er wird lokal in SQLite gespeichert.
KI-Score fehlt	Sensitivitaetsdaten fuer die Passes pruefen; LlmAnalysisService liest SENSITIVITY_DETAIL.
Dukascopy-Daten fehlen fuer einzelne Tage	Download-Scan nutzen; Wochenenden und Feiertage koennen keine Ticks liefern.
CSV-Import in MT5 klappt nicht	Custom Symbol Name, Digits, Skriptdeployment und MT5-Log pruefen.
Parameter wirken falsch exportiert	SET-Merge, EaParameterManager und Pass-Parameterwerte kontrollieren.
Score wirkt kontraintuitiv	ScoreWeights pruefen; hohe Gewichte fuer FW-Trades und SampleSize koennen kleine Gewinnwunder bremsen.
UI zeigt alten Zustand	Workflow-State aus SQLite wurde geladen. Workflow resetten oder passende History wiederherstellen.
MCP findet Datenbank nicht	Backtester einmal starten, damit %USERPROFILE%\.mt5_backtester/history.db angelegt wird.
Reports sind optisch unvollstaendig	Report-Generator und Quellreport pruefen; Parser kann nur vorhandene MT5-Daten extrahieren.

Anhang E: Glossar

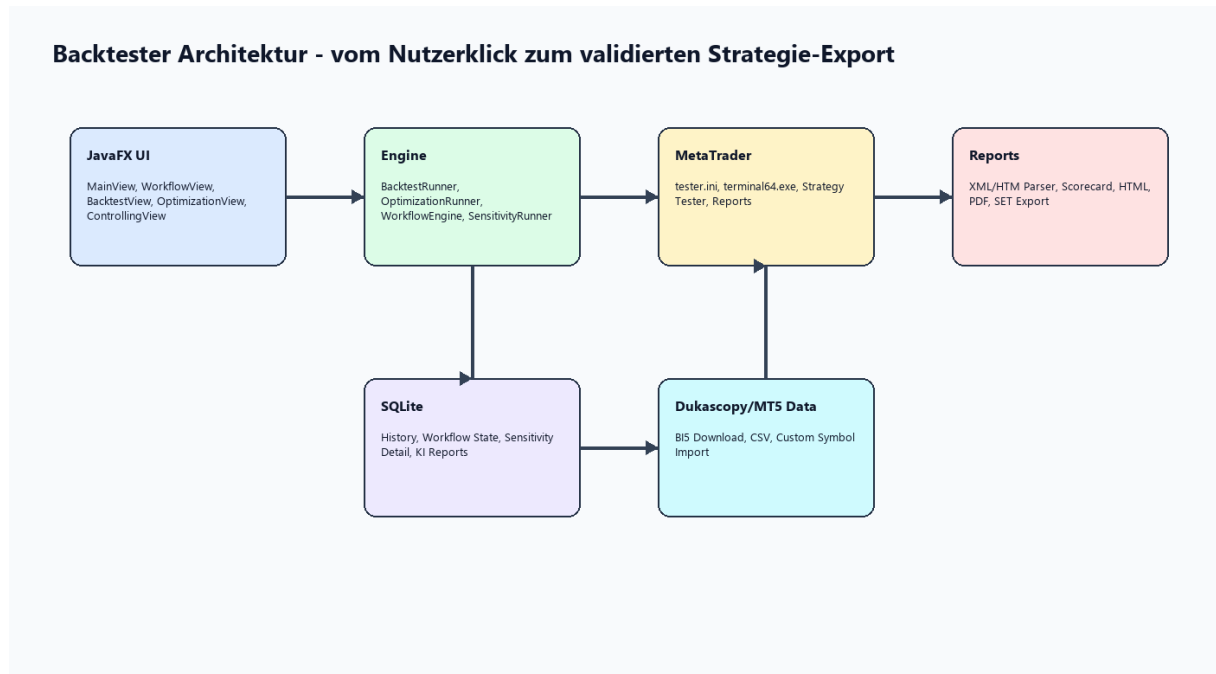
Das Glossar uebersetzt zentrale Begriffe aus Trading, Statistik, MetaTrader und Projektarchitektur in kurze Arbeitsdefinitionen.

Begriff	Bedeutung
Backtest	Historische Simulation einer Handelsregel mit bekannten Marktdaten.
Forward-Test	Von MT5 abgetrennter Teil des Optimierungszeitraums, der zur ersten Plausibilisierung dient.
Out-of-Sample	Daten, die nicht in Optimierung oder Auswahl eingeflossen sind.
In-Sample	Datenbereich, in dem Parameter gesucht oder angepasst werden.
Curve Fitting	Uebermaessige Anpassung an historische Zufallsdetails.
Overfitting	Statistische Ueberanpassung, die in neuen Daten typischerweise einbricht.
Walk-Forward	Wiederholte Optimierung und Validierung ueber rollierende Zeitfenster.
Holdout	Zurueckgelegtes Datenfenster fuer finale Validierung.
Monte Carlo	Simulation vieler Varianten, um Zufallseinfluesse sichtbar zu machen.
Expert Advisor	Automatisierte Handelsstrategie in MetaTrader.
SET-Datei	MetaTrader-Parameterdatei fuer Expert Advisors.
tester.ini	Konfigurationsdatei, mit der MetaTrader per Kommandozeile gesteuert wird.
Optimization Pass	Eine getestete Parameterkombination im MT5-Optimizer.
CombinedPass	Projektobjekt, das Backtest- und Forward-Pass zusammenfuehrt.
Profit Factor	Verhaeltnis von Bruttogewinn zu Bruttoverlust.
Recovery Factor	Verhaeltnis von Gewinn zu maximalem Drawdown.
Sharpe Ratio	Risikoadjustierte Renditekennzahl.
Drawdown	Rueckgang vom Equity-Hoch zum folgenden Tief.
Expected Payoff	Durchschnittliches Ergebnis pro Trade.
Coefficient of Variation	Relative Streuung; im Projekt als CV fuer Parametersensitivitaet genutzt.
Plateau	Parameterbereich, in dem Ergebnisse stabil bleiben.
Peak	Einzelner Spitzenwert ohne stabile Nachbarschaft.
Cliff	Klippenfoermiger Einbruch bei kleiner Parameterveraenderung.
Diversity Filter	Auswahlmechanismus, der zu aehnliche Strategien reduziert.
KI-Gate	Filter, der sehr fragile KI-bewertete Kandidaten aussortiert.
Step 7	Finale Validierung auf unberuehrtem OOS-Fenster nach Portfolio-Auswahl.
Best-Ordner	Exportziel fuer besonders gute und nach Validierung akzeptierte Strategien.
Workflow State	Persistierter Zustand der siebenstufigen Pipeline.
SENSITIVITY_DETAIL	Normalisierte SQLite-Tabelle fuer CV, Kurvendaten und Verdicts.
OpenRouter	API-Schicht fuer den Zugriff auf LLM-Modelle.
MCP	Model Context Protocol; ermoeeglicht KI-Tools lesenden Zugriff auf Backtester-Daten.
Dukascopy BI5	Komprimiertes Tickdatenformat von Dukascopy.
Custom Symbol	In MT5 importiertes Symbol mit eigenen historischen Daten.
Portable Mode	MT5-Startmodus mit lokaler Datenhaltung im Installationskontext.
Process Guard	Schutzlogik gegen haengende oder stale MetaTrader-Prozesse.
Virtual Desktop	Start von MetaTrader auf einem zweiten Desktop, damit die UI frei bleibt.
Report Parser	Code, der HTM/XML-Reports in strukturierte Ergebnisobjekte uebersetzt.
ScoreWeights	Single Source of Truth fuer Score-Gewichte im Projekt.
WorkflowEngine	Zentrale State Machine der Anti-Curvefitting-Pipeline.
BacktestRunner	Runner fuer einzelne MT4/MT5-Backtests.
OptimizationRunner	Runner fuer MT5-Optimierungen und Forward-Auswertung.
SensitivityRunner	Runner fuer Parameter-Sweeps und CV-Berechnung.

Begriff	Bedeutung
RobustnessRunner	Runner fuer Robustheitsscans ueber Parameter und Zeitverschiebungen.
DatabaseManager	SQLite-Zugriffsschicht und Migrationspunkt des Projekts.

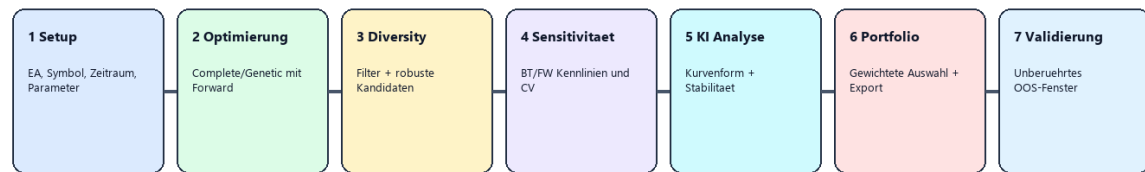
Anhang F: Screenshot-Galerie

Die Galerie zeigt zuerst die fuer dieses Buch erzeugten Erklaergrafiken und danach die wichtigsten vorhandenen Projektbilder. Sie lockert das Buch auf und dient zugleich als visueller Index der Architektur und Benutzeroberflaeche.



Grafikindex: Architekturuebersicht des Backtester-Projekts.

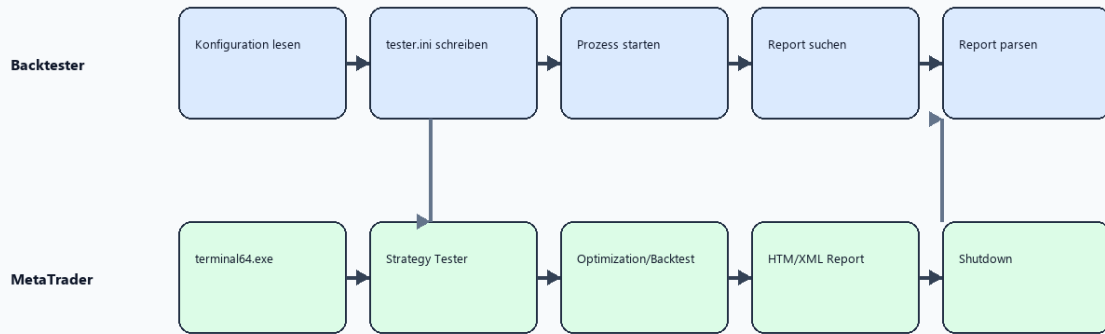
7-Schritt Anti-Curvefitting Workflow



Fachlicher Kern: Der Forward-Test ist ein Auswahlkriterium. Erst Schritt 7 liefert die echte, nachgelagerte Out-of-Sample-Schaetzung.

Grafikindex: 7-Schritt Anti-Curvefitting Workflow.

MT5 Prozessablauf im Backtester



Grafikindex: MT5-Prozessablauf.

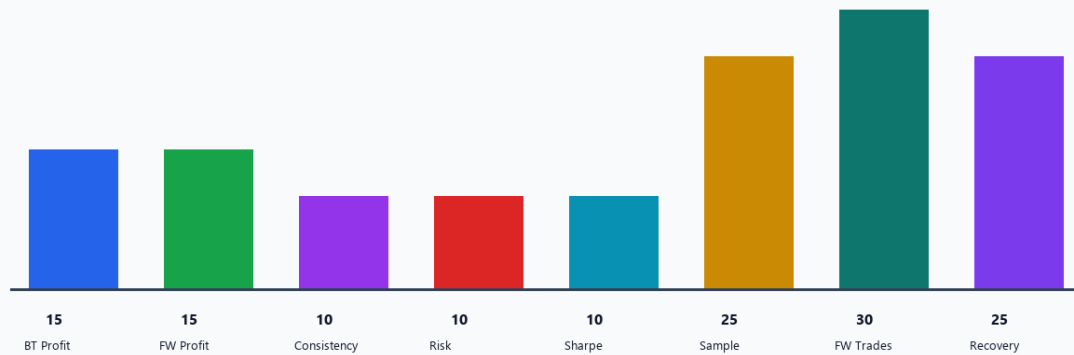
Persistenzmodell: SQLite als Gedächtnis der Anwendung



Grafikindex: Persistenzmodell der Anwendung.

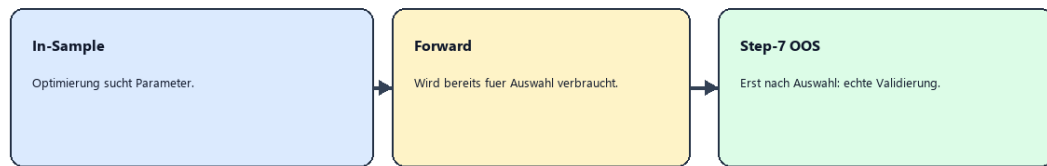
Scorecard-Modell: gemessene Saeulen statt synthetischer Schoenheit

Alle Gewichte werden normalisiert. Die Saeulen beruhen auf realen MT5-Kennzahlen wie Profit, Trades, Drawdown, Sharpe und Recovery.



Grafikindex: Scorecard-Modell.

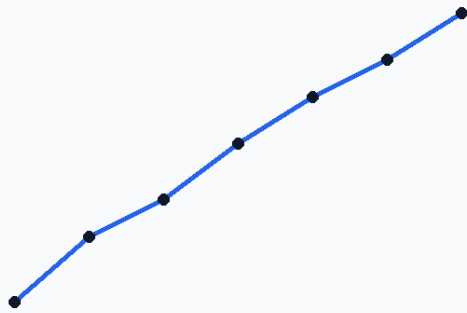
Warum Step 7 noetig ist



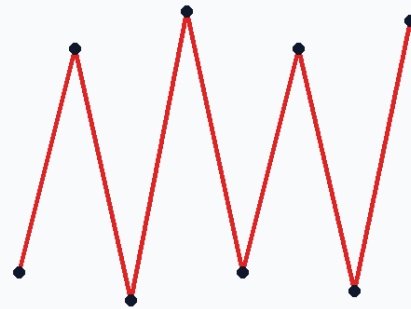
Wenn tausende Paesse getestet werden, findet man fast immer Gewinner im Forward-Fenster. Das ist noch kein Beweis fuer Live-Robustheit.

Grafikindex: Step-7-OOS-Gate.

Curve Fitting: Signal, Rauschen und die falsche Sicherheit



Robuster Trend: wenige Regeln, stabile Nachbarschaft



Ueberangepasste Kurve: perfekte Vergangenheit, fragile Zukunft

Grafikindex: Curve-Fitting-Erklärergrafik.



Abbildung: Gesamtplattform des Backtesters.

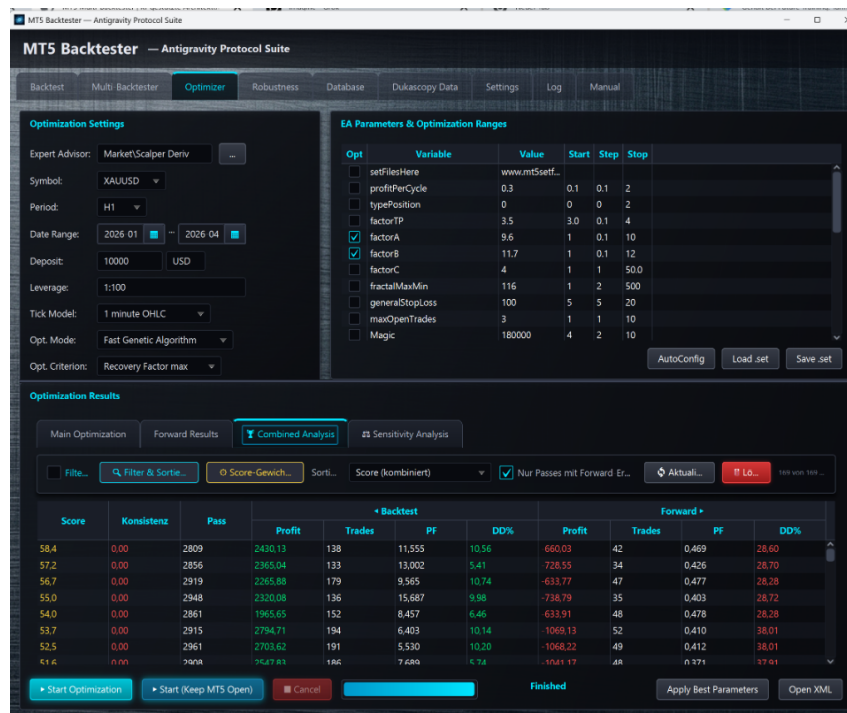


Abbildung: Optimizer-Arbeitsbereich.

Score-Gewichtung konfigurieren

Score-Gewichtung

Jeder Parameter wird relativ zum anderen gewichtet...

BT Profit

10%

FW Profit

20%

Konsistenz FW/BT

20%

FW Profit Factor

20%

Drawdown-Strafe

30%

FW Trade Count

80%

Erholungsfaktor

50%

Σ = 230 (wird normalisiert)

Automatische Schutzwelle: FW-Trades unter median/2 erhalten zusätzlich eine multiplikative Strafe (ma...

Voreinstellungen:

Low / Zahm

Med / Ausgewogen

High / Streng

Zurücksetzen

✓ Übernehmen & Schließen

Abbrechen

Abbildung: Score-Gewichtung und Ranking.

Mastering the Backtester

Seite 45



Abbildung: Sensitivitätsanalyse und Robustheitskurven.

MT5 Backtester — Antigravity Protocol Suite

Backtest

Multi Backtest

Optimizer

Robustness

Database

Dukascopy Data

Settings

Log

Manual

Optimization Settings

Expert Advisor:

Market/Scalper Deriv

Symbol:

XAUUSD

Period:

H1

Date Range (12 Months):

2025-04

2026-04

Deposit:

10000

USD

Leverage:

1:100

Tick Model:

1 minute OHLC

Opt. Mode:

Fast Genetic Algorithm

Opt. Criterion:

Return Factor max

EA Parameters & Optimization Ranges

Opt	Variable	Value	Start	Step	Stop
☐	setFilesHere	www.mt5setf...			
☐	Buy & Sell	0.1	0.1	2	
☐	Buy & Sell	3.0	0.2	4	
☐	Buy & Sell	1	0.2	10	
☐	Buy & Sell	7	0.2	12	
☐	Buy & Sell	1	1	50.0	
☐	Buy & Sell	5	2	500	
☐	Buy & Sell	100	5	20	
☒	maxOpenTrades	3	1	6	
☐	Magic	180000	4	2	10

AutoConfig Load .set Save .set

Optimization Results

Main Optimization

Forward Results

Combined Analysis

+ Selected

Sensitivity Analysis

Pass	Name	BT Profit	FW Profit	BT CV (worst)	FW CV (worst)	Status
15545	15545+15545	22870.90	38823.17	88.48 %	57.91 %	Completed
14907	14907+14907	2354.11	35175.73	200.00 %	107.99 %	Completed
15823	15823+15823	7762.93	19088.45	105.61 %	78.02 %	Completed

Start Sensitivity Analysis

Cancel

Ergebnisse löschen

KI-Analyse starten

KI-Einstellungen

Sensitivity Analysis completed.

Abbildung: KI-Analyse der Strategie-Stabilität.

Mastering the Backtester

Seite 47

Pass	Status	Score	CV worst	Fragile	Fazit
2175	✓ Robust	95	33.12%	0	Sehr stabil, exzellent
2039	✓ Robust	94	29.15%	0	Sehr stabil, exzellent
1769	✓ Robust	93	30.33%	0	Sehr stabil, exzellent
2252	✓ Robust	92	37.03%	0	Sehr stabil, exzellent
2351	✓ Robust	91	37.03%	0	Sehr stabil, exzellent
1065	✓ Robust	90	32.22%	0	Sehr stabil, exzellent
1028	✓ Robust	85	48.02%	0	Stabil, gute Performance
1074	⚠ Fragil	70	43.72%	0	Akzeptabel, FW schwächer
2653	⚠ Fragil	65	41.99%	0	Akzeptabel, FW schwächer
909	⚠ Fragil	68	42.52%	0	Akzeptabel, FW schwächer
728	⚠ Fragil	60	45.07%	0	Akzeptabel, FW schwächer

Abbildung: KI-Bewertungstabelle.

Score ①	Konsis... ①	KI ①	Pass	◀ Backtest					Forward ▶				
				Profit	Trades ▼	PF	DD%	Erholung	Profit	Trades	PF	DD%	Erholung
72,10	0.84	94 / 100	2039	1747,53	191	7,51	8,68	1,92	2082,87	211	2,58	18,92	0,95
72,40	0.83	92 / 100	2252	1733,16	190	7,83	8,44	1,96	1985,27	205	2,65	17,75	0,99
72,40	0.83	91 / 100	2351	1732,83	190	7,82	8,44	1,96	1985,27	205	2,65	17,75	0,99
72,80	0.85	90 / 100	1065	1752,14	196	7,92	8,44	1,98	2056,24	208	2,72	17,68	1,02
74,90	0.91	93 / 100	1769	1755,31	201	6,04	7,50	2,24	2290,02	217	3,27	17,45	1,14
74,80	0.91	95 / 100	2175	1933,88	212	8,29	8,46	2,16	2426,81	248	3,05	17,26	1,21
72,50	0.98	85 / 100	1028	2709,69	154	4,58	16,24	1,45	2785,68	159	3,92	17,67	1,34
77,00	1.07	70 / 100	1074	4689,23	150	2,73	19,23	2,07	5063,26	135	3,43	15,20	2,21
79,20	1.12	60 / 100	728	9093,32	163	2,82	16,45	2,43	9767,47	139	5,17	19,09	2,82
88,90	0.86	65 / 100	2653	3033,45	158	30,63	6,02	4,43	3370,13	169	31,86	10,29	2,72
86,50	1.06	68 / 100	909	1765,44	181	12,69	5,32	3,17	2242,17	197	7,63	7,24	2,71

Abbildung: Auswahl und Export bester Strategien.

Maximum Gewinn aufeinanderfolgender Gewinntrades (Anzahl): **195.08 (6)** Maximum Verlust aufeinanderfolgender Verlusttrades (Anzahl): **-4**
 Durchschnitt Gewinntrades in Folge: **6** Durchschnitt Verlusttrades in Folge:

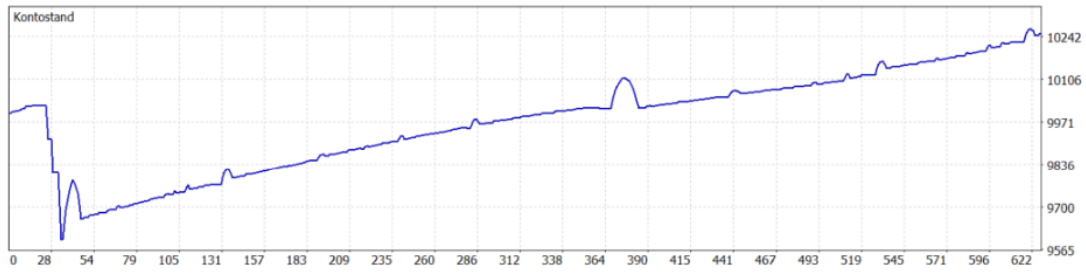


Abbildung: Multi-Backtester-Konfiguration.



Abbildung: Multi-Backtester-Ergebnisse.

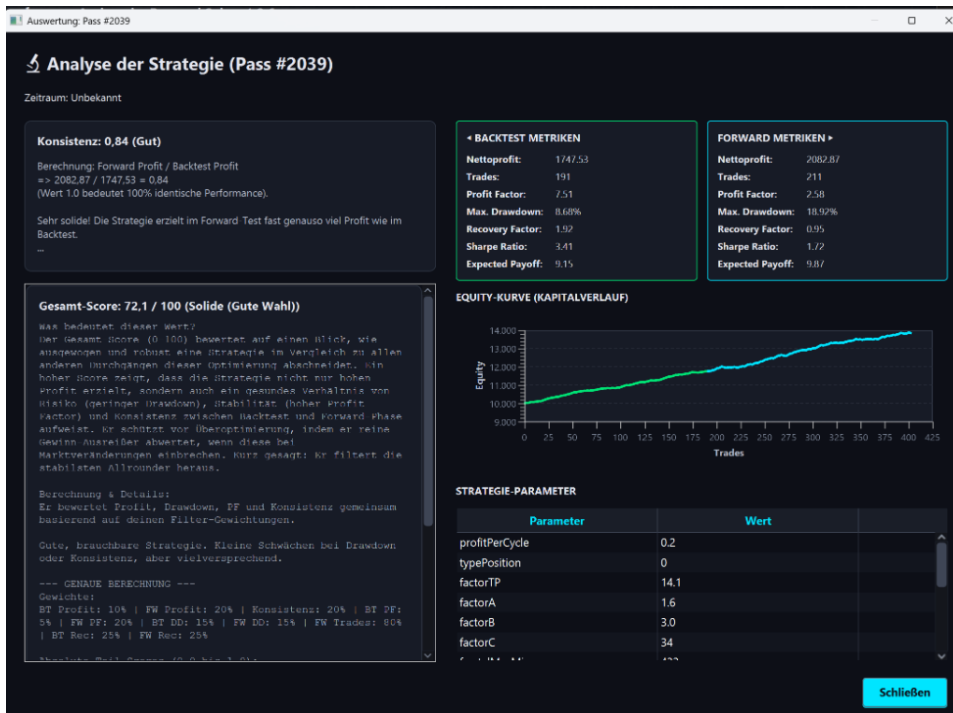


Abbildung: Strategie-Detailanalyse.

Quellen und Bildnachweis

Webquellen wurden fuer die Lehrbuchteile zu Backtesting, Overfitting, Walk-Forward-Analyse und Out-of-Sample-Validierung herangezogen. Projektaussagen wurden aus dem lokalen Repository abgeleitet.

Quelle	Titel	URL
QuantStart	Successful Backtesting of Algorithmic Trading Strategies - Part I	https://www.quantstart.com/articles/Successful-Backtesting-of-Algorithmic-Trading-Strategies-Part-I/
Investopedia	Backtesting and Forward Testing: The Importance of Correlation	https://www.investopedia.com/articles/trading/10/backtesting-walkforward-important-correlation.asp
AlgoTrading101	Backtesting Biases and Risks	https://algotrading101.com/wiki/backtesting-biases-and-risks/
Surmount	Walk-Forward Analysis vs. Backtesting	https://surmount.ai/walk-forward-analysis-vs-backtesting-pros-cons-best-practices
QuantInsti	Walk-Forward Optimization	https://blog.quantinsti.com/walk-forward-optimization-introduction/

Bildnachweis

Bild	Herkunft
architecture.png	Fuer dieses Buch generierte Erklergrafik
workflow.png	Fuer dieses Buch generierte Erklergrafik
mt5_process.png	Fuer dieses Buch generierte Erklergrafik
database.png	Fuer dieses Buch generierte Erklergrafik
scorecard.png	Fuer dieses Buch generierte Erklergrafik
oos_gate.png	Fuer dieses Buch generierte Erklergrafik
curve_fitting.png	Fuer dieses Buch generierte Erklergrafik
images/backtester_platform.png	Abbildung: Gesamtplattform des Backtesters.
images/backtester_optimizer.png	Abbildung: Optimizer-Arbeitsbereich.
images/backtester_score_weighting.png	Abbildung: Score-Gewichtung und Ranking.
images/backtester_sensitivity.png	Abbildung: Sensitivitetsanalyse und Robustheitskurven.
images/backtester_ki_analysis.png	Abbildung: KI-Analyse der Strategie-Stabilitat.
images/backtester_ki_evaluation_table.png	Abbildung: KI-Bewertungstabelle.
images/backtester_best_strategies.png	Abbildung: Auswahl und Export bester Strategien.
images/multi-backtester-config.png	Abbildung: Multi-Backtester-Konfiguration.
images/multi-backtester-results.png	Abbildung: Multi-Backtester-Ergebnisse.
images/backtester_strategy_detail_analysis.png	Abbildung: Strategie-Detailanalyse.